

Fundamentals of Software Testing

Strategies and Techniques

White Box Testing (Structural)

- **Definition:** Testing based on analysis of internal logic, design, and code.
- **Coverage-based Testing - Possible Criteria**
 - **Statement Coverage:** Execute every statement at least once.
 - **Branch Coverage:** Exercise true/false outcomes of every decision.
 - **Condition Coverage:** Exercise true/false outcomes of every condition in any branch.
 - **Path Coverage:** Execute all linearly independent paths (LIPs).
- **Control Flow Graphs (CFG):**
 - Nodes represent statements;
 - Edges represent the flow of control.
 - Used to design test cases that cover paths through the code.

White Box Testing

Statement, Branch, Condition, and Path Coverage

Understanding White Box Testing

- Testing based on internal code structure
- Requires knowledge of logic, control flow, and conditions
- Used in:
 - Unit Testing
 - Code Optimization
 - Logic Verification
- Coverage types:
 - Statement Coverage
 - Branch Coverage
 - Condition Coverage
 - Path Coverage

Statement Coverage

- Ensures every statement executes at least once
- Simplest form of white-box testing
- Does not guarantee logical correctness

C++ Program: Statement Coverage

```
#include <iostream>
using namespace std;

void checkNumber(int x) {
    if (x > 0)
        cout << "Positive\n";
        cout << "Done\n";
}

int main() {
    checkNumber(5);
}
```

Test Case	Input	Coverage Achieved
TC1	5	All statements executed

Table: Statement Coverage Test Case

C++ Program: Branch Coverage

- Ensures both true and false branches execute
- Covers all decision outcomes

```
void checkEvenOdd(int x) {  
    if (x % 2 == 0)  
        cout << "Even\n";  
    else  
        cout << "Odd\n";  
}  
  
int main() {  
    checkEvenOdd(2);  
}
```

Test Case	Input	Branch Covered
TC1	2	True branch
TC2	3	False branch

Table: Branch Coverage Test Cases

C++ Program: Condition Coverage

- Each condition evaluated as true and false

```
void checkConditions(int a, int b) {  
    if (a > 0 && b > 0)  
        cout << "Both-Positive\n";  
    else  
        cout << "At-least-one-non-positive\n";  
}  
  
int main() {  
    checkConditions(1,1);  
}
```

Test Case	a	b	Result
TC1	1	1	T,T
TC2	-1	1	F,T
TC3	1	-1	T,F

Table: Condition Coverage Test Cases

C++ Program: Path Coverage

- Covers all possible execution paths
- Most comprehensive testing technique
- May lead to path explosion

```
void findType(int x) {  
    if (x > 0) {  
        if (x % 2 == 0)  
            cout << "Positive-Even\n";  
        else  
            cout << "Positive-Odd\n";  
    } else {  
        cout << "Non-positive\n";  
    }  
}  
  
int main() {  
    findType(2);  
}
```

Path Coverage Test Cases

Test Case	Input	Path Covered
TC1	2	Positive Even
TC2	3	Positive Odd
TC3	-1	Non-positive

Comparison of Coverage Techniques

Coverage Type	Strength	Limitation
Statement	Simple	Misses logic errors
Branch	Decision coverage	Ignores sub-conditions
Condition	Logical accuracy	More test cases
Path	Comprehensive	Complex, exponential paths

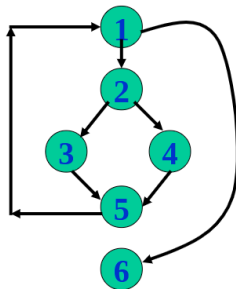
Remember:

- White-box testing validates internal logic
- Different coverage criteria provide different levels of assurance
- Combination of techniques ensures robust testing
- Path coverage is powerful but not always practical

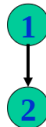
How to Draw Control Flow Graph?

- Number all the statements of a program.
- Numbered statements:
 - Represent nodes of the control flow graph.
- An edge from one node to another node exists:
 - If execution of the statement representing the first node
 - Can result in transfer of control to the other node.

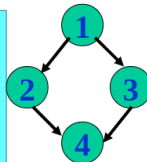
```
int f1(int x,int y){  
  1 while (x != y){  
    2 if (x>y) then  
      3   x=x-y;  
      4   else y=y-x;  
    5 }  
  6 return x;  
}
```



- Sequence:
 - 1 a=5;
 - 2 b=a*b-1;



- Selection:
 - 1 if(a>b) then
 - 2 c=3;
 - 3 else c=5;
 - 4 c=c*c;



- Iteration:
 - 1 while(a>b){
 - 2 b=b*a; b=b-1;}
 - 3 c=b+d;

7

Cyclomatic Complexity

- **McCabe's Metric:** Measures the structural complexity of program code.
- **Formulas for $V(G)$:**
 - $V(G) = E - N + 2$ (where E = edges, N = nodes).
 - $V(G)$ = number of bounded areas + 1.
- **Significance:** Provides an upper bound on the number of independent paths and test cases needed.
- **Guideline:** Complexity higher than 10 should generally be avoided for maintainability.

- **Concept:** A form of static analysis focusing on the *definition* and *usage* of variables.
- **Variable States:** Defined (d), Reference (r), and Undefined (u).
- **Anomalies:**
 - **ur-anomaly:** Reading an undefined variable.
 - **du-anomaly:** Variable assigned a value that becomes undefined without being used.
 - **dd-anomaly:** Variable receiving a second value before the first is used.
- **DU Pairs:** Defined at node i and used at node j .

Regression Testing

- Does not belong to either unit test, integration test, or system test.
 - Instead, it is a separate dimension to these three forms of testing.
- Regression testing is the running of test suite:
 - after each change to the system
 - after each bug fix.
- Ensures that no new bug has been introduced due to the change or the bug fix.
- Regression tests assure:
 - the new system's performance is at least as good as the old system.
 - Always used during incremental system development.

- **Definition:** Testing to ensure that recent changes (bug fixes or new features) do not negatively affect existing functionality.
- **Tasks in Regression Testing:**
 - ① **Test Revalidation:** Checking which old tests are still valid.
 - ② **Test Selection (RTS):** Identifying tests that traverse modified portions of code.
 - ③ **Test Minimization:** Removing redundant tests based on coverage criteria.
 - ④ **Test Prioritization:** Ordering tests based on criteria like maximum coverage to find errors sooner.

- **GUI Facts:** GUIs can constitute high percentage (~60%) of code, and testing accounts for 40-60% of development costs.
- **Challenges:** Event-driven architecture and a vast space of possible user actions (clicks, pixel-level interactions).
- **Web Architecture:** Heterogeneous systems involving various browsers, application servers, and databases.
- **Automated Testing with Selenium:**
 - **Selenium IDE:** Firefox/Chrome extension for record and replay.
 - **Selenium WebDriver:** Programmatic control of browsers, supporting multiple languages (Java, Python, etc.).

Other Testing Techniques

Stress Testing

- Stress testing (aka endurance testing) impose abnormal input to stress the capabilities of the software.
 - Input data volume, input data rate, processing time, utilization of memory, etc. are tested beyond the designed capacity.
- Evaluates system performance - when stressed for short periods of time.
- Stress tests are black box tests:
 - Designed to impose a range of abnormal and even illegal input conditions so as to stress the capabilities of the software.
- Stress testing usually involves an element of time or size,
 - Such as the number of records transferred per unit time,
 - The maximum number of users active at any time, input data size, etc.
- Stress testing may not be applicable to all types of systems.

Stress Testing - Example Use Cases...cont'd

- If the requirements is to handle a specified number of users, or devices:
 - Stress testing evaluates system performance when all users or devices are busy simultaneously.
- If an operating system is supposed to support 15 multiprogrammed jobs,
 - The system is stressed by attempting to run 15 or more jobs simultaneously.
- A real-time system might be tested
 - To determine the effect of simultaneous arrival of several high-priority interrupts.

Volume Testing

- Addresses handling large amounts of data in the system:
 - Whether data structures (e.g. queues, stacks, arrays, etc.) are large enough to handle all possible situations.
 - Fields, records, and files are stressed to check if their size can accommodate all possible data volumes.

Configuration Testing

- Analyze system behavior in various hardware and software configurations specified in the requirements.
- Sometimes systems are built in various configurations for different users, for instance, a minimal system may serve a single user, other configurations for additional users.

Compatibility Testing

- These tests are needed when the system interfaces with other systems:
 - Check whether the interface functions as required.
- If a system is to communicate with a large database system to retrieve information:
 - A compatibility test examines speed and accuracy of retrieval.

Recovery Testing

- These tests check response to presence of faults or to the loss of data, power, devices, or services, subject system to loss of resources
- Check if the system recovers properly.

Maintenance Testing

- Diagnostic tools and procedures help find source of problems.
- It may be required to supply
 - memory maps, diagnostic programs,
 - traces of transactions, circuit diagrams, etc.
- Verify that all required artifacts for maintenance exist, and they function properly

Documentation tests

- Check that required documents exist and are consistent:
 - user guides, maintenance guides, technical documents etc.
- Sometimes requirements specify format and audience of specific documents.
- Documents are evaluated for compliance

Usability tests

- All aspects of user interfaces are tested:
 - Display screens, messages
 - report formats, navigation and selection problems etc.

Environmental test

- These tests check system's ability to perform at the installation site.
- Requirements might include tolerance for heat, humidity, chemical presence, portability, electrical/magnetic fields, power disruption, etc.
- Critical use cases - setting up cloud infrastructure, data centers etc.

Test Summary Report

- Generated towards the end of testing phase.
- Covers each subsystem:
 - A summary of tests which have been applied to the subsystem.
- Specifies:
 - how many tests have been applied to a subsystem,
 - how many tests have been successful,
 - how many have been unsuccessful, and the degree to which they have been unsuccessful,
- For e.g. whether a test was an outright failure or whether some expected results of the test were actually observed.

What it is?

- Debugging is the specific process of **fixing a bug** in the software.
- It is defined as the sequence of identifying, analyzing, and removing errors.

When Debugging Occurs

- This activity begins **after** the software fails to execute properly during testing.
- It concludes only after the problem is solved and the software is successfully re-tested.

Why Debugging is Tedious

- It is considered an extremely complex task.
- Errors must be meticulously resolved at every stage of the process.

Aspect	Testing	Debugging
Purpose	Find defects in software	Fix the defects
Who Performs	Testers/QA team	Developers/Programmers
Tools	Selenium, JUnit, LoadRunner	IDE debuggers, GDB, Valgrind
Process	Planning → Execution → Reporting	Identify → Analyze → Fix → Verify
Stage	Before release / during development	After a bug is found
Outcome	Bug reports	Fixed and working code

- Software testing is an essential, creative, and challenging part of development.
- No amount of testing can guarantee a program is 100% error-free; it only shows the *presence* of bugs.
- **Effective Strategy:**
Combine Black Box testing (for functionality) and White Box testing (for structural integrity) to build confidence in the software.